
Guia de Despliegue para Pruebas del Cliente – SIGAI-SES

Securitas Colombia S.A.

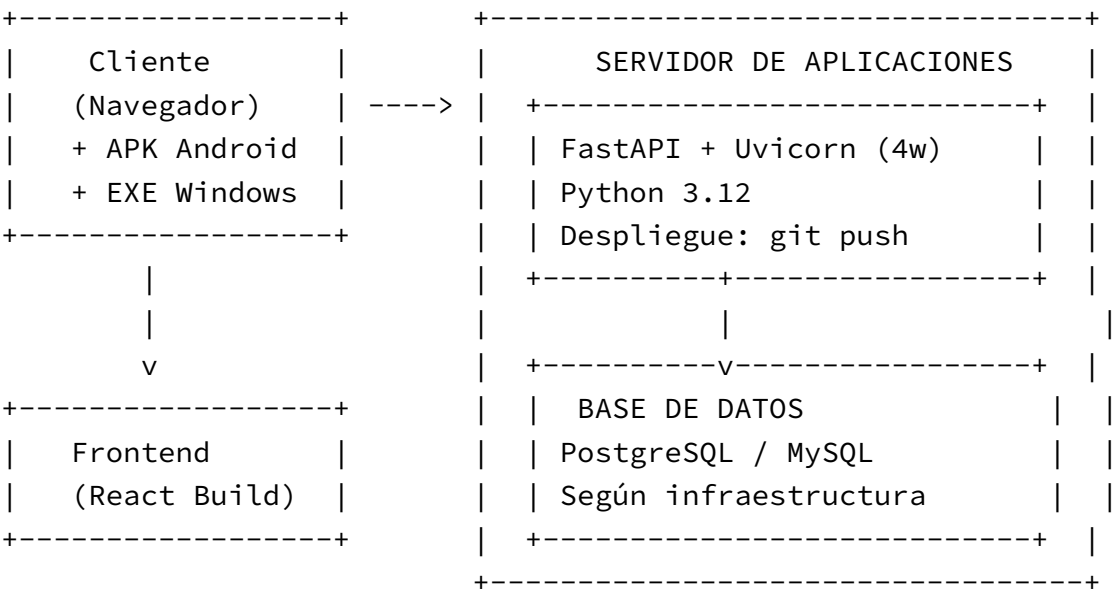


Guia de Despliegue para Pruebas del Cliente – SIGAI-SES

VERSION	1.0.0	STATUS	PRODUCCIÓN	LICENSE	PROPRIETARY
STACK	FASTAPI → SQL				

[!TIP] **Objetivo:** Publicar SIGAI-SES en un servidor corporativo o cloud, con actualizaciones desde VS Code mediante `git push`.

Arquitectura del Despliegue



Nota: El stack de despliegue (servidor, base de datos, dominio) es definido por el área de TI de Securitas según sus políticas y estándares internos. El sistema es compatible con PostgreSQL, MySQL y MariaDB.

Indice de Pasos

#	Paso	Servicio	Tiempo
1	Subir código a GitHub	GitHub	~10 min
2	Configurar base de datos	PostgreSQL / MySQL	~15 min
3	Configurar variables de entorno	Servidor	~5 min
4	Desplegar Backend	Servidor corporativo / cloud	~15 min
5	CI Pipeline (GitHub Actions)	GitHub	~5 min
5	Probar el sistema	–	~5 min
6	Dar acceso al cliente	–	~5 min
7	Actualizaciones desde VS Code	VS Code	~1 min

PASO 1: Subir el código a GitHub

[!IMPORTANT] El código debe estar en un repositorio GitHub. El servidor corporativo o cloud se conectará al repositorio para despliegues.

Repositorio actual: <https://github.com/mrwil82/SIGAI-SES>

Verificar que el `.gitignore` excluye archivos sensibles

```
.env  
Backend/.env  
Frontend/.env  
*.log  
__pycache__/  
node_modules/  
.venv/  
Frontend/dist/  
Backend/app/static/avatars/
```

PASO 2: Configurar Base de Datos

[!NOTE] El sistema soporta **PostgreSQL, MySQL y MariaDB**. La elección depende de la infraestructura definida por TI.

Opcion A: PostgreSQL

```
DATABASE_URL=postgresql+asyncpg://USUARIO:CONTRASENA@HOST:5432/NOMBRE_BD  
DATABASE_URL_SYNC=postgresql://USUARIO:CONTRASENA@HOST:5432/NOMBRE_BD
```

Opcion B: MySQL / MariaDB

```
DATABASE_URL=mysql+aiomysql://USUARIO:CONTRASENA@HOST:3306/NOMBRE_BD  
DATABASE_URL_SYNC=mysql+pymysql://USUARIO:CONTRASENA@HOST:3306/NOMBRE_BD
```

Inicializar tablas

Al iniciar el backend por primera vez, las tablas se crean automáticamente. Para aplicar migraciones manuales:

```
cd Backend  
alembic upgrade head
```

[!CAUTION] Guarda las credenciales de conexión. Las necesitarás en el PASO 3.

PASO 3: Desplegar Backend en Servidor

[!TIP] El backend se despliega en el servidor definido por TI (corporativo, nube privada, o servicio cloud).

3.1 Opciones de despliegue

Opcion	Descripcion
Servidor corporativo (on-premise)	Ubuntu Server + Nginx + Systemd + Let's Encrypt (ver 06_GUIA_ON_PREMISE .md)
Nube privada	AWS, Azure, GCP con Docker o maquina virtual
Cloud gratuito para pruebas	Servicios cloud gratuitos

3.2 Variables de entorno requeridas

Variable	Descripcion	Ejemplo
DATABASE_URL	Conexion asincrona a BD	postgresql+asyncpg://user:pass
DATABASE_URL_SYNC	Conexion sincrona (reportes)	postgresql://user:pass@host:5
SECRET_KEY	Clave secreta JWT	clave_secreta_segura_123
ALGORITHM	Algoritmo JWT	HS256
ACCESS_TOKEN_EXPIRE_MINUTES	Minutos hasta expiracion	480
ADMIN_EMAIL	Email admin inicial	admin@securitas.com
ADMIN_PASSWORD	Password admin inicial	Admin123!
CORS_ALLOWED_ORIGINS	Origenes permitidos (coma)	https://midominio.com,http://

3.3 Iniciar el servidor

```
cd Backend
uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

3.4 Verificar

```
# Abre en el navegador:
http://TU_SERVIDOR:8000/docs
# Debes ver la documentacion Swagger de la API
```

PASO 4: CI Pipeline (GitHub Actions)

[!NOTE] El pipeline se ejecuta automáticamente en cada `git push` y ejecuta: - **backend-tests**: 32 tests (pytest con SQLite async) - **frontend-lint**: ESLint con `--max-warnings 300`

Workflow definido en `.github/workflows/main.yml`

```
name: CI Pipeline
on: [push]
jobs:
  backend-tests:
    runs-on: ubuntu-latest
    env:
      DATABASE_URL: "sqlite+aiosqlite:///memory:"
      SECRET_KEY: "test-secret-key"
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: "pip"
      - run: pip install -r Backend/requirements.txt
      - run: pytest Backend/tests/

  frontend-lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: "npm"
          cache-dependency-path: Frontend/package-lock.json
      - run: npm ci
        working-directory: Frontend
      - run: npm run lint
        working-directory: Frontend
```

[!TIP] Para ver el estado del pipeline: <https://github.com/mrwil82/SIGAI-SES/actions>

PASO 5: Probar

- ☒ Abre la URL del servidor en el navegador
 - ☒ Login: admin@securitas.com / Admin123!
 - ☒ Dashboard carga con KPIs
 - ☒ Prueba crear un item, un cliente, una garantia
 - ☒ Genera un acta de entrega con PDF
 - ☒ Cambia el tema en Configuracion
 - ☒ Sube tu foto de avatar
 - ☒ Verifica que los tests pasan en GitHub Actions
-

PASO 6: Builds Ejecutables

APK Android

```
cd Frontend
npm run cap:apk
# Genera: Frontend/android/app/build/outputs/apk/debug/app-debug.apk (~4.4
  ↳ MB)
```

EXE Windows (Portable)

```
cd Backend
pyinstaller backend.spec
# Genera: Backend/dist/SIGAI-SES.exe (~67 MB)
```

Instalador Windows

```
# Usar Inno Setup con dist_installer/setup.iss
# Genera: dist_installer/SIGAI-SES-Setup-1.0.0.exe
```

PASO 7: Actualizaciones desde VS Code

[!TIP] Para hacer cambios y que se reflejen automaticamente en produccion:

```
cd C:\Users\ASUS\Desktop\PASANTIA\Proyecto_SES
# Editar archivos en VS Code...
# Luego:

git add .
git commit -m "Descripcion del cambio"
git push
```

Componente	Tiempo estimado	Gatillo
Backend	~3 minutos	git push+redploy
Frontend	~2 minutos	git push+redploy
CI Pipeline	~4 minutos	git push

[!IMPORTANT] El metodo de actualizacion depende del servidor (webhook, polling, o manual).
En git push se recomienda configurar un webhook.

Comandos Utiles

Click para expandir comandos

Ver logs del backend

```
# Si usa systemd:
journalctl -u sigai-ses -f

# Si usa Docker:
docker logs -f sigai-ses-api
```

Verificar estado de la BD

```
# PostgreSQL:
psql -h HOST -p 5432 -U USUARIO -d NOMBRE_BD

# MySQL:
mysql -h HOST -u USUARIO -p
```

Rollback (si algo se rompe)

```
# Revertir migracion Alembic:
cd Backend
alembic downgrade -1

# Revertir commit en git:
git revert HEAD --no-edit
git push
```

Solucion de Problemas

Problema	Solucion
Frontend no carga	Verificar que VITE_API_BASE_URL apunta al backend correcto
Error 500 en API	Revisar logs del servidor
Base de datos no conecta	Verificar DATABASE_URL, firewalls, y credenciales
“value too long for type VARCHAR(255)”	Ejecutar: ALTER TABLE usuarios ALTER COLUMN avatar_url TYPE TEXT;
Login falla	Verificar SECRET_KEY y DATABASE_URL en el servidor
Los cambios no se ven	Verificar que el redeploy se completo correctamente

Problema	Solucion
404 en avatars	Los avatars viejos en disco no existen; subir avatar desde Configuracion

Progreso del Despliegue

Paso 1 [=====] 100% - Código en GitHub
Paso 2 [=====] 80% - Base de datos configurada
Paso 3 [=====] 80% - Backend desplegado
Paso 4 [=====] 100% - CI Pipeline funcional
Paso 5 [=====] 80% - Pruebas
Paso 6 [=====] 80% - Builds ejecutables
Paso 7 [=====] 100% - Actualizaciones automaticas

Documento actualizado: Julio 2026 – v1.0 Repositorio: <https://github.com/mrwil82/SIGAI-SES>